END-TO-END MLOps

Heart Disease UCI Classification

Experimental Learning Assignment

Deliverables covered

• Data acquisition and exploratory analysis

• Feature engineering and model comparison

• MLflow experiment tracking

• Reproducible model packaging

• CI/CD and automated tests

• FastAPI + Docker model serving

• Minikube deployment manifests

• Prometheus metrics and logging

• Documentation and evidence checklist

Student: [Enter name]

Course / institution: [Enter details]

Date: [Enter submission date]

GitHub repository: [Add URL after publishing]

Important: educational software demonstration only; not a medical diagnostic system.

1. Executive Summary

This project implements a reproducible machine learning lifecycle for binary classification on the processed Cleveland subset of the UCI Heart Disease dataset. It combines data acquisition, EDA, train-fitted preprocessing, candidate model comparison, cross-validation, MLflow tracking, automated tests, a FastAPI inference service, Docker packaging, local Kubernetes deployment, request logs, and Prometheus metrics.

Logistic Regression and Random Forest are compared. A stratified holdout set is reserved for final evaluation, and stratified five-fold cross-validation with grid search selects hyperparameters. The chosen estimator is selected by mean CV ROC-AUC. The serialized artifact includes both preprocessing and the estimator.

Measured results — fill after training

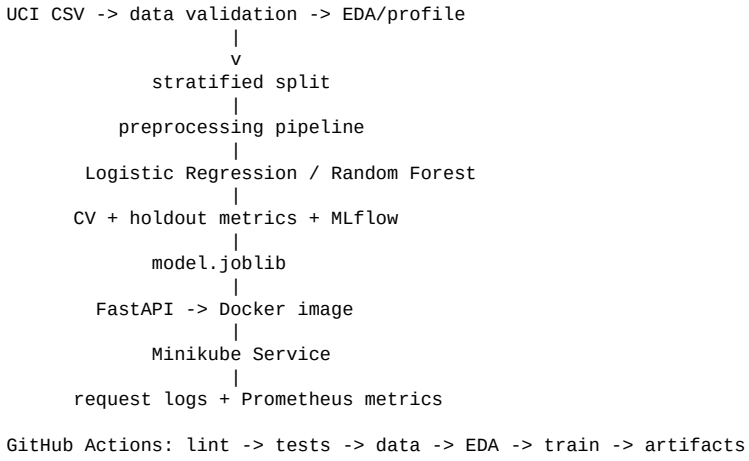
Model	CV ROC-AUC	Accuracy	Precision	Recall	Test ROC-AUC
Logistic Regression	[generated]	[generated]	[generated]	[generated]	[generated]
Random Forest	[generated]	[generated]	[generated]	[generated]	[generated]

Do not enter assumed metrics. Use artifacts/training_summary.json generated by the training command.

2. Problem Statement and Architecture

The goal is to demonstrate the end-to-end engineering workflow required to move a model from raw data to a monitored API. The API accepts JSON containing the 13 predictor values and returns a binary prediction, class label, maximum class probability, and positive-class probability. This is a coursework implementation rather than a clinically validated risk estimator.

Architecture



The main folders are src/ for implementation, tests/ for unit tests, .github/workflows/ for CI, deployment/ for Kubernetes manifests, monitoring/ for Prometheus configuration, artifacts/ for generated outputs, and screenshots/ for genuine evidence.

3. Data Acquisition and Cleaning

The project uses the classic processed Cleveland file from the UCI Heart Disease dataset. It is commonly documented as 303 observations with 13 predictors and a target. Fields cover age, sex, chest pain, blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise angina, ST depression, slope, major vessels, and thal-related category.

src/data.py downloads the UCI processed file, parses '?' as missing, validates expected columns, coerces values to numeric, and converts the original target to a binary label: 0 remains 0 and values 1–4 become 1. A public mirror is used as a fallback. The target mapping is explicitly documented because it changes the original target coding.

```
python -m src.data
python -m src.eda
```

The EDA script emits data_profile.json with row and column counts, missingness, class counts, and descriptive statistics. It also generates age distributions grouped by target, class balance, and a correlation heatmap. Inspect the generated files before writing data-specific conclusions.

Data quality notes

Missing-value handling is performed inside the pipeline rather than by fitting transformations on the full dataset. This preserves validation integrity. The source data is small and historical; it may not represent contemporary populations or clinical practice.

4. Exploratory Data Analysis

EDA provides a descriptive view of the target distribution, missingness, predictor ranges, and pairwise linear correlation. Histograms reveal spread and skew. A class-count plot highlights possible imbalance. A correlation heatmap helps identify correlated numeric-coded fields, but correlation does not establish causation.

Figure 1 — distributions and target balance

[INSERT artifacts/eda_distributions.png after running python -m src.eda]

Figure 2 — feature correlation heatmap

[INSERT artifacts/correlation_heatmap.png after running python -m src.eda]

Report the observed class counts and missing values from artifacts/data_profile.json. Do not assume there are no missing values; the original processed Cleveland file includes missing markers in some fields.

5. Feature Engineering and Model Development

A ColumnTransformer combines numeric and categorical transformations. Numeric features use median imputation and StandardScaler. Categorical-coded features use most-frequent imputation and one-hot encoding with unknown categories ignored. This reflects the fact that category codes should not automatically be interpreted as continuous distances.

The target is removed from X. The data is split into an 80% training partition and 20% held-out test partition with stratification and random seed 42. Because preprocessing is part of the scikit-learn Pipeline, imputers, scalers, and encoders are fitted only on the training fold during cross-validation.

Candidate models and tuning

Logistic Regression is the baseline, with balanced class weights and a grid over regularization parameter C. Random Forest is a nonlinear ensemble with balanced class weights and a grid over maximum depth and minimum leaf size. Both candidates use StratifiedKFold with five folds and ROC-AUC scoring.

Final held-out metrics include accuracy, precision, recall, F1, and ROC-AUC. ROC-AUC measures ranking across thresholds; precision and recall describe different error trade-offs. No single metric is sufficient to establish clinical validity.

Actual model result

[INSERT selected model, CV ROC-AUC, and test metrics from artifacts/training_summary.json.]

6. Experiment Tracking and Model Packaging

MLflow experiment heart-disease-uci records one run for each candidate model. Logged parameters include the model name, seed, test fraction, and selected hyperparameters. Metrics include mean CV ROC-AUC and held-out accuracy, precision, recall, F1, and ROC-AUC. Artifacts include the classification report, evaluation figures, and the fitted pipeline.

```
mlflow ui --backend-store-uri ./mlruns --host 127.0.0.1 --port 5000
```

Figure 3 — MLflow UI: [INSERT genuine screenshot showing both runs and metrics].

The selected full preprocessing-plus-estimator pipeline is written to artifacts/model.joblib. This keeps inference transformations consistent with training. requirements.txt specifies supported package ranges; training_summary.json records seeds, split sizes, selected model, hyperparameters, and metrics.

For stronger reproducibility, pin transitive dependencies in a lock file, record a source-data checksum, pin container base images by digest, and version the model artifact. Only load serialized Python objects from trusted sources and compatible environments.

7. CI/CD and Automated Testing

The GitHub Actions workflow triggers on pushes to main/master, pull requests, and manual dispatch. It installs dependencies, runs Ruff linting and Pytest, downloads the dataset, generates EDA, trains models, and uploads artifacts and MLflow runs. This demonstrates continuous integration and repeatable training; it does not publish an image or automatically deploy a public service.

Tests cover dataset schema, binary target conversion, missing marker parsing, model pipeline fit/predict and probability output, unsupported model names, API health, model-not-loaded behavior, and Prometheus metrics availability.

Evidence

Figure 4 — pytest output: [INSERT actual terminal screenshot].

Figure 5 — GitHub Actions: [INSERT actual successful workflow run and uploaded artifacts].

A production extension could separate fast unit tests from slower training integration tests, scan the container image, publish signed images to a registry, and require an approval before deployment.

8. API, Docker, and Minikube Deployment

FastAPI exposes GET / for service metadata, GET /health for readiness, GET /metrics for Prometheus, and POST /predict for schema-validated inference. Input constraints reject values outside configured ranges. The response includes predicted class, label, model confidence, and probability of the positive class. Confidence is not a calibrated medical confidence measure.

```
python -m src.data
python -m src.train
docker build -t heart-disease-mlops:local .
docker run --rm -p 8000:8000 heart-disease-mlops:local
```

The Docker image includes the application, dependencies, and trained artifact. Interactive API docs are available at http://localhost:8000/docs. The model must be trained before the image is built.

Minikube

```
minikube start
minikube image load heart-disease-mlops:local
kubectl apply -f deployment/deployment.yaml
kubectl rollout status deployment/heart-disease-api
kubectl get pods,svc
kubectl port-forward service/heart-disease-api 8000:8000
```

Figure 6 — Docker container and health check: [INSERT genuine screenshot].

Figure 7 — API prediction: [INSERT actual request and response].

Figure 8 — Minikube pod/service: [INSERT actual kubectl output].

The manifest includes readiness/liveness probes and CPU/memory requests and limits. It is a local educational deployment, not a high-availability production cluster. Public deployment would require TLS, authentication, secrets management, network policy, registry hosting, and security review.

9. Monitoring, Limitations, and Conclusion

The API logs request method, path, status, and latency, as well as prediction completion without intentionally logging the full input payload. Prometheus counters expose request totals by method, endpoint, and status; a histogram measures request latency. The provided Compose monitoring configuration scrapes the API's /metrics endpoint.

Figure 9 — Prometheus target and metric query: [INSERT genuine screenshot of target status and api_requests_total or api_request_latency_seconds].

The current monitoring is a basic service-level demonstration. It does not detect feature drift, label drift, or changes in model quality. Production monitoring should include privacy-aware input validation metrics, distribution checks, model version tags, alerting, retention policies, and a governed model re-evaluation process.

Evidence checklist

■ EDA plots generated and inspected

■ MLflow runs captured

■ Pytest output captured

■ GitHub Actions run captured

■ Docker API verified

■ Prometheus target and query verified

■ Minikube deployment and health endpoint verified

■ GitHub repository URL added

■ Short walkthrough video recorded and linked

Conclusion: the repository demonstrates the core MLOps stages from dataset acquisition through local Kubernetes serving and basic monitoring. The appropriate success criteria are reproducibility, correctness, and operational evidence—not an unsupported claim of clinical accuracy.

References

• UCI Heart Disease dataset: https://archive.ics.uci.edu/dataset/45/heart+disease

• scikit-learn: https://scikit-learn.org/stable/

• MLflow: https://mlflow.org/docs/latest/

• FastAPI: https://fastapi.tiangolo.com/

• Docker: https://docs.docker.com/

• Kubernetes: https://kubernetes.io/docs/

• Prometheus: https://prometheus.io/docs/